Praval

Building Agent Systems with Praval 0.8.1

Rajesh Sampathkumar

Book Edition 1.1, July 2026

Contents

Preface

This book describes Praval 0.8.1. It is a maintained reference for the exact published wheel, not a general promise about later versions. Book Edition 1.1 replaces the earlier October 2025 edition, which described Praval 0.7.6.

Praval is a Python framework for model-backed agents and agent teams. It separates two concerns. Agent teams coordinate through Agent handlers, Reef, and Spores. Individual agents execute model requests through `ModelRuntime` and provider adapters. This separation is the central technical change in the 0.8 line.

The examples in this edition come from the certified examples and notebook course shipped with Praval 0.8.1. Each Python block has a validation mode. Complete offline examples run against the exact wheel. Self-contained fragments compile against that wheel. Examples that need credentials or services are shown only when the text states why they cannot run in the offline book check.

This book distinguishes framework behavior from application design advice. When a statement depends on a provider, model, service, or optional package, the dependency is stated beside it. Measured results come from the Praval paper-validation harness. They are not inferred from diagrams or examples.

Chapter 1

Decide whether specialist agents fit the problem

An agent is a named unit that can receive input, keep local state, call a model, use tools, and produce output. A specialist agent has a narrower responsibility than the application as a whole. Examples include extracting facts, checking evidence, planning a response, or approving a sensitive operation.

Specialization can make ownership and testing clearer. It can also add model calls, messages, failure points, and operational work. A team of agents is useful only when those costs are justified.

1.1 Good reasons to split work

Use separate agents when at least one of these conditions holds:

- Different stages need different tools, credentials, or provider policies.
- Independent tasks can run concurrently and have a clear join condition.
- One stage must check or approve another stage's output.
- Different teams own different parts of the system.
- A message boundary makes failure, audit, or retry behavior easier to state.

Do not split a short, sequential transformation only to make the system look like a multi-agent application. A function or one Agent is often the clearer choice.

1.2 Costs that must be budgeted

Every added agent may create a model call. It may also add serialization, queueing, tracing, deployment, and review costs. Reef can schedule independent handlers concurrently, but concurrency does not make dependent work independent. If stage B needs the complete result from stage A, the critical path still contains both stages.

An agent team also needs explicit answers to these questions:

- Which message ends the workflow?
- How are duplicate or late messages handled?
- Which failures are returned as domain results?
- Which failures should stop the process?
- Which operations need human approval?
- Which components own cleanup?

The framework provides delivery and lifecycle mechanisms. The application still owns its domain protocol.

1.3 A small design test

The following pure function is a useful review aid before writing agent code. It does not decide architecture automatically. It makes the main costs visible.

```
def specialist_split_is_justified(
    independent_work: bool,
    distinct_tools: bool,
    separate_approval: bool,
    strict_sequential_dependency: bool,
) -> bool:
    benefits = sum((independent_work, distinct_tools, separate_approval))
    return benefits >= 2 and not strict_sequential_dependency


assert specialist_split_is_justified(True, True, False, False)
assert not specialist_split_is_justified(False, False, False, True)
```

Treat the result as a prompt for review. Latency, correctness, and operating cost must still be measured for the actual workload.

Chapter 2

Agent, Reef, Spore, and ModelRuntime

Praval uses four core terms.

An `Agent` owns a name, model configuration, conversation history, registered tools, and optional memory or human intervention state. A decorated agent also has a Python handler that receives Spores.

A `Spore` is the message envelope used for agent-to-agent communication. It contains routing identity, a JSON-oriented knowledge dictionary, timestamps, priority, reply metadata, and optional references.

`Reef` routes Spores to subscribed handlers. It supports direct delivery, broadcasts, request and reply metadata, completion tracking, and shutdown. The default Reef is in process. A RabbitMQ backend is available for distributed delivery.

`ModelRuntime` validates and normalizes model execution. It accepts common request options, checks provider capabilities, calls an adapter, runs registered tools when requested, and returns common response or event types.

2.1 Two planes

The coordination plane is Agent handler to Reef to Spore to Agent handler. The execution plane is Agent API to `ModelRuntime` to provider adapter to model endpoint. A decorated Agent can use both planes. A direct Agent can use only the execution plane. A Reef handler can also perform deterministic Python work without calling a model.

This distinction helps locate failures. A missing subscription is a coordination problem. An unsupported image request is an execution capability problem. A rejected tool call belongs to the Agent and HITL policy boundary.

2.2 Coral is a name, not technical evidence

The names Praval and Reef were chosen to suggest small participants exchanging information in a shared environment. The metaphor does not prove emergence, resilience, decentralization, or scaling. Those properties require explicit semantics and evidence.

Praval draws on actor-like isolation and asynchronous messaging, but it does not claim formal Actor Model conformance (Hewitt et al. 1973; Agha 1986). Reef uses publish-subscribe concepts, whose delivery and coupling properties depend on the actual implementation (Eugster et al. 2003).

2.3 Direct Reef delivery

This complete offline example creates its own Reef, subscribes a handler, sends one Spore, waits for completion, and shuts the Reef down.

```
from praval.core.reef import Reef


received = []
reef = Reef()
reef.subscribe("reviewer", lambda spore: received.append(spore.knowledge))
reef.send("author", "reviewer", {"type": "draft", "draft_id": "d-1"})

assert reef.wait_for_completion(timeout=5)
assert received == [{"type": "draft", "draft_id": "d-1"}]
assert reef.shutdown(timeout=5)
```

The completion wait covers work scheduled through that Reef. It does not turn the in-memory Reef into a durable queue or replay log.

Chapter 3

Praval through 0.7.22 and the 0.8.1 transition

Praval 0.8.1 extended an existing coordination framework. Understanding the previous stable release prevents two common errors. The first is attributing all current behavior to `ModelRuntime`. The second is assuming that 0.7.22 already had the 0.8 execution contracts.

3.1 The coordination foundation

By the 0.7 series, Praval had direct and decorated Agents, Reef delivery, and Spores. It also had registered tools, completion tracking, optional memory, storage providers, observability, transport adapters, and a RabbitMQ Reef backend. These features formed the operating model around agent coordination.

Version 0.7.22 was the last stable 0.7 release. It was released on February 21, 2026, from commit `ee056118f9daa444166bd09148a3ca42649ccfd3` (Sampathkumar 2026a).

3.2 The 0.7.22 HITL boundary

Version 0.7.22 added agent-gated human intervention for protected provider tool calls. An agent could opt in to HITL. Praval stored the intervention and suspended run in SQLite. An operator could approve the call, edit its arguments, reject it, and resume the run after restarting the process.

The release also defined a fail-closed rule. If a tool required approval and the agent had HITL disabled, Praval raised `HITLConfigurationError`. It did not execute the tool silently.

The 0.7.22 implementation connected this path to the OpenAI, Anthropic, and Cohere tool loops supported by that release. It did not include `ModelRuntime`, `EmbeddingRuntime`, provider profiles, MCP clients, Spore V2 content parts, or `PravalApp`.

3.3 What 0.8.1 added

Praval 0.8.1 retained the coordination interfaces and added a provider-neutral execution plane (Sampathkumar 2026b). Its main additions and expansions include:

- `ModelRuntime` and common model request, response, event, usage, tool, and error types.
- `EmbeddingRuntime` with explicit provider, model, and dimension configuration.
- Provider and model profiles with fail-fast capability validation.
- Synchronous, asynchronous, streaming, structured, reasoning, multimodal, tool, and media request paths.
- Gemini and local OpenAI-compatible provider support.
- Tools-only MCP clients for stdio and Streamable HTTP.

- Spore V2 content, correlation, causation, trace, run, and idempotency fields.
- `PravalApp` as an explicit cleanup owner for retained Agents and a Reef.
- Exact-wheel release certification and versioned learning resources.

The two-plane description records this version boundary. It does not mean that coordination and model execution never interact. A handler can call a model, and `ModelRuntime` can pause a tool call through HITL.

3.4 Verify the installed artifact

The published Praval 0.8.1 wheel has SHA-256 `70b0220a2ced6c0bd066423566c4e1811caa9015b128604d2bc5b3c8d57379c5`. The book validator installs that file into a temporary environment and checks the import path before it runs examples.

```
import pathlib

import praval


package_path = pathlib.Path(praval.__file__).resolve()
assert praval.__version__ == "0.8.1"
assert package_path.name == "__init__.py"
```

Chapter 4

Define Agents and own their lifecycle

Use a direct `Agent` when application code initiates a model request. Use the `@agent` decorator when a Python handler should receive Spores from Reef. The two forms share the underlying Agent execution path, but they serve different entry points.

4.1 A bounded decorated-agent workflow

`responds_to` filters on `spore.knowledge["type"]`. The initial message below starts one handler. Its explicit broadcast starts the second handler. The workflow ends because the second handler does not emit another Spore.

```
from praval import (
    agent,
    broadcast,
    get_provider_registry,
    get_reef,
    start_agents,
)
from praval.models import ProviderCapabilities


results = []


class HandlerOnlyProvider:
    provider_name = "handler-only"
    capabilities = ProviderCapabilities()

    def __init__(self, config):
        self.config = config

    def close(self):
        pass


get_provider_registry().register_provider(
    "handler-only",
    HandlerOnlyProvider,
    default_model="offline",
)
```

```
@agent(
    "normalizer",
    provider="handler-only",
    model="offline",
    responds_to=["raw_record"],
    auto_broadcast=False,
)
def normalize_record(spore):
    value = str(spore.knowledge["value"]).strip().lower()
    broadcast({"type": "record_ready", "value": value})


@agent(
    "collector",
    provider="handler-only",
    model="offline",
    responds_to=["record_ready"],
    auto_broadcast=False,
)
def collect_record(spore):
    results.append(spore.knowledge["value"])


start_agents(
    normalize_record,
    collect_record,
    initial_data={"type": "raw_record", "value": "  READY  "},
)
reef = get_reef()
assert reef.wait_for_completion(timeout=5)
assert results == ["ready"]
reef.shutdown()
```

The application supplies the message schema. Praval validates the Spore envelope, not the meaning of `raw_record` or `record_ready`. The example registers the same handler-only provider used by the offline certification pattern because every underlying Agent needs a provider. The deterministic handlers do not send a model request.

4.2 Direct model APIs

`chat()` returns a string for compatibility. `generate()` returns a `ModelResponse` with content, finish state, provider metadata, usage, and tool information. `agenerate()` is its asynchronous counterpart. `stream()` and `astream()` emit normalized `ModelEvent` values.

Provider and model support differs. A method being present does not mean that every provider profile supports every option.

```
import asyncio

from praval import Agent

agent = Agent(
    "reviewer",
    provider="openai",
    model="gpt-5.4-mini",
    system_message="Check claims against the supplied evidence.",
)
```

```
text = agent.chat("Review the draft.")
response = agent.generate("Return a structured review.")


async def review_async() -> str:
    async_response = await agent.agenerate("Review this asynchronously.")
    return async_response.content


asyncio.run(review_async())
agent.close()
```

This fragment needs a configured OpenAI key and an available model. It compiles during book validation but is not sent to a provider.

4.3 `chat` and `achat` inside handlers

The top-level helpers use the current decorated Agent. Call them only while a handler is executing. `achat` is appropriate for an async handler.

```
from praval import achat, agent, chat


@agent("sync-writer", responds_to=["write"])
def sync_writer(spore):
    return {"type": "written", "text": chat(spore.knowledge["prompt"])}


@agent("async-writer", responds_to=["write_async"])
async def async_writer(spore):
    text = await achat(spore.knowledge["prompt"])
    return {"type": "written", "text": text}
```

These calls use the Agent's configured provider, model, tools, history, and runtime policy.

4.4 `PravalApp`

`PravalApp` retains Agents and a Reef so one owner can close them. It does not isolate the process-wide provider registry, and it does not redirect every global Reef helper into the app's Reef. Use it for lifecycle ownership, not as a dependency injection container.

```
from praval import PravalApp


with PravalApp() as app:
    reviewer = app.create_agent(
        "reviewer",
        provider="anthropic",
        model="claude-sonnet-5",
    )
    response = reviewer.generate("Review the release note.")
    print(response.content)
```

When the context exits, the app closes its retained Agents and owned Reef. External storage providers and MCP clients still need their own cleanup.

Chapter 5

Reef topologies, Spore V2, and distributed delivery

Reef supports several message shapes. A direct send names one destination. A broadcast reaches matching subscribers. A request carries reply metadata. A reply links to the request. Pipelines and fan-out or fan-in arrangements are application protocols built from these operations.

5.1 Choose a topology from dependencies

Use a pipeline when each stage depends on the previous stage. Use fan-out when branches are independent. Use fan-in when one handler can determine that all required branch results have arrived. Use request and reply when the sender needs a correlated response.

Praval does not require an application-level workflow orchestrator for these registered message topologies. Reef remains shared coordination infrastructure. A RabbitMQ deployment also depends on a broker.

5.2 Spore V2

Spore V2 preserves the legacy `knowledge` body and adds JSON-safe content and reference fields. Raw bytes are not valid Spore content. Store large or binary data elsewhere and send a `DataReference`, or use a content part that the selected provider accepts.

```
from datetime import datetime

from praval import ContentPart, Spore, SporeType


spore = Spore(
    id="book-spore-v2",
    spore_type=SporeType.KNOWLEDGE,
    from_agent="collector",
    to_agent="reviewer",
    knowledge={"type": "evidence_ready"},
    created_at=datetime.now(),
    schema_version="2.0",
    content_parts=[ContentPart.text_part("Evidence summary")],
    data_references=["files://file_system/evidence/report.json"],
    correlation_id="review-42",
    causation_id="ingest-17",
)
```

```
restored = Spore.from_json(spore.to_json())
assert restored.knowledge == {"type": "evidence_ready"}
assert restored.content_parts[0]["text"] == "Evidence summary"
assert restored.correlation_id == "review-42"
```

Treat a received Spore as immutable application data. Some helper methods return derived Spores. The dataclass itself is not a cryptographic or language-enforced immutability boundary.

5.3 Completion and shutdown

`wait_for_completion()` waits for work registered with the Reef or channel. It returns `False` on timeout. A handler that continually broadcasts new work can keep completion pending. Every workflow therefore needs a terminal condition.

Shutdown is idempotent in 0.8.1. It closes handlers, executor resources, and the configured backend. Idempotent cleanup does not imply message persistence, redelivery, or exactly-once processing.

5.4 RabbitMQ backend

RabbitMQ is the distributed Reef backend. The backend needs the optional dependencies, a reachable broker, queue naming, and process-level lifecycle management. The protocol exercised by the service tests is AMQP-backed RabbitMQ behavior, not a universal broker contract (Committee 2012; Broadcom 2024).

```
import asyncio

from praval.core.reef import Spore, SporeType
from praval.core.reef_backend import RabbitMQBackend


async def publish(spore: Spore) -> None:
    backend = RabbitMQBackend()
    await backend.initialize(
        {
            "url": "amqp://guest:guest@127.0.0.1:5672/",
            "exchange_name": "praval.book",
        }
    )
    try:
        await backend.send(spore, "agent.reviewer")
    finally:
        await backend.shutdown()


message = Spore.create(
    spore_type=SporeType.KNOWLEDGE,
    from_agent="writer",
    to_agent="reviewer",
    knowledge={"type": "draft_ready"},
)
asyncio.run(publish(message))
```

This fragment compiles offline. Running it requires RabbitMQ and the relevant optional package. The application must decide how to handle broker outages, duplicates, slow consumers, and poison messages.

Chapter 6

Provider-neutral model and embedding execution

`ModelRuntime` is the boundary between Agent methods and provider adapters. It builds a `ModelRequest`, merges safe profile defaults, resolves declared capabilities, validates the request, applies bounded retry policy, invokes the adapter, runs client tools when requested, and returns a `ModelResponse` or `ModelEvent` stream.

Provider-neutral does not mean provider-identical. Each provider and model has its own endpoints, limits, finish reasons, tool semantics, media types, and usage reports. Praval normalizes the fields that its contracts cover and keeps provider-specific options behind an explicit field.

6.1 A deterministic fake provider

The following example comes from the maintained offline runtime example. It tests the runtime contract without credentials or network access.

```
from praval.core.agent import AgentConfig
from praval.model_runtime import ModelRuntime
from praval.models import ModelEvent, ModelResponse, ProviderCapabilities


class FakeProvider:
    provider_name = "book-fake"
    capabilities = ProviderCapabilities(
        streaming=True,
        native_streaming=True,
        structured_outputs=True,
    )

    def invoke(self, request):
        return ModelResponse(
            content='{"status": "checked"}',
            provider=self.provider_name,
            model=request.model,
        )

    def stream(self, request):
        yield ModelEvent(type="delta", delta="checked")
        yield ModelEvent(
            type="final",
```

```
            response=ModelResponse(
                content="checked",
                provider=self.provider_name,
                model=request.model,
            ),
        )


runtime = ModelRuntime(
    provider=FakeProvider(),
    provider_name="book-fake",
    config=AgentConfig(provider="book-fake", model="book-model"),
)
response = runtime.invoke(
    messages=[{"role": "user", "content": "Check the record."}],
    response_schema={
        "type": "object",
        "properties": {"status": {"type": "string"}},
        "required": ["status"],
    },
)
assert response.content == '{"status": "checked"}'
assert [event.type for event in runtime.stream(
    messages=[{"role": "user", "content": "Stream the status."}]
)] == ["start", "delta", "final"]
```

6.2 Requests and responses

Use `ModelMessage` and `ContentPart` when an application needs a typed request. Most application code can call the Agent APIs and let the runtime build those objects.

`ModelResponse` contains normalized content, provider and model identifiers, finish state, usage, tool calls, reasoning metadata, and provider metadata when available. A provider may omit usage or other optional fields. Code must not assume that every provider returns every field.

6.3 Streaming

The normalized event sequence can include `start`, `delta`, `tool_call_delta`, `tool_call`, `tool_result`, `usage`, `error`, and `final`. Native streaming depends on the adapter and profile. A fallback may emit a complete response as one delta followed by a final event. This is still a stream-shaped contract, but it is not token-by-token provider streaming.

6.4 Structured output and reasoning

Structured output is constrained by the provider and endpoint. Praval passes a normalized schema and returns JSON text in `ModelResponse.content`. Applications should parse and validate that text when schema compliance is a hard requirement. JSON Schema terms follow the registered schema specification (Wright et al. 2022).

Reasoning options use `ReasoningConfig`, but provider fields and accepted values differ. A profile may reject reasoning even when another model from the same provider accepts it.

```
from praval import Agent, ReasoningConfig


agent = Agent("planner", provider="openai", model="gpt-5.4-mini")
response = agent.generate(
```

```
    "Return a short task list.",
    response_schema={
        "type": "object",
        "properties": {
            "tasks": {
                "type": "array",
                "items": {"type": "string"},
            }
        },
        "required": ["tasks"],
    },
    reasoning=ReasoningConfig(effort="low"),
)
print(response.content)
agent.close()
```

This fragment needs an available OpenAI model that supports both options. A capability error should be handled as a configuration result, not retried against an unrelated provider.

6.5 Provider profiles and fail-fast checks

A `ProviderProfile` records the provider, model, endpoint, capabilities, defaults, limits, and downgrade policy known at release time. It is not a live provider catalog. Providers can change availability after the wheel is published.

Unsafe request options such as API keys and authorization headers are rejected at the runtime boundary. An inaccurate custom profile can still overstate an endpoint's capabilities, so custom profiles need tests.

```
from praval import Agent, ModelMessage, ModelRequest


agent = Agent("local", provider="ollama", model="llama3")
request = ModelRequest(
    provider="ollama",
    model="llama3",
    messages=[ModelMessage(role="user", content="hello")],
)

capabilities = agent.runtime.resolve_capabilities(request)
agent.runtime.validate_request(request)
print(capabilities.streaming)
agent.close()
```

Local presets for Ollama, vLLM, LM Studio, llama.cpp, and generic OpenAI-compatible servers are conservative. Tools, structured output, reasoning, and media depend on the server version, model, and launch options.

6.6 Embeddings

`EmbeddingRuntime` is separate from chat configuration. The selected provider, model, and dimensions must agree with the vector collection. Changing an incompatible embedding configuration requires a new or rebuilt collection.

```
from praval import EmbeddingRuntime


runtime = EmbeddingRuntime(provider="local", dimensions=32)
response = runtime.embed(["reef coordination", "runtime execution"])
```

```
assert len(response.embeddings) == 2
assert all(len(vector) == response.dimensions for vector in response.embeddings)
```

The local path is useful for deterministic tests. It does not establish that local vectors have the semantic quality required by a specific application.

6.7 Multimodal content and request-based voice

Use `ContentPart` for typed text, image, file, audio, or video inputs. The runtime validates the content type against the selected profile. A valid content part can still fail if the provider rejects the media, URL, size, or model combination.

OpenAI-backed Agents expose bounded `transcribe()` and `speak()` requests. Praval 0.8.1 does not provide a persistent realtime audio session.

```
from praval import Agent, ContentPart


vision = Agent("vision", provider="gemini", model="gemini-3.5-flash")
response = vision.generate(
    [
        ContentPart.text_part("Describe the diagram."),
        ContentPart.image_url_part("https://example.com/diagram.png"),
    ]
)
print(response.content)
vision.close()

voice = Agent("voice", provider="openai", model="gpt-5.4-mini")
audio = voice.speak("The validation run is complete.")
print(audio.content_type)
voice.close()
```

Running this fragment requires credentials, available models, network access, and provider-supported media. The live capability protocol records those external conditions separately.

Chapter 7

Tools, MCP, and durable human intervention

A tool is a typed operation that a model may request. The model chooses the tool and arguments. Praval validates and executes the registered Python handler. Tool use therefore crosses a trust boundary. Validate arguments in the handler, restrict credentials, set timeouts, and require approval for sensitive effects.

ReAct and related work provide useful context for interleaving model reasoning and actions (Yao et al. 2023). That literature does not prove that any particular tool loop is safe.

7.1 Decorated tools

The `@tool` decorator registers a Python function and its metadata. A decorated Agent can list the tool by name.

```
from praval import (
    agent,
    get_provider_registry,
    get_reef,
    start_agents,
    tool,
)
from praval.models import ProviderCapabilities


results = []


class HandlerOnlyProvider:
    provider_name = "handler-only"
    capabilities = ProviderCapabilities()

    def __init__(self, config):
        self.config = config

    def close(self):
        pass


get_provider_registry().register_provider(
    "handler-only",
```

```
    HandlerOnlyProvider,
    default_model="offline",
)


@tool("add_numbers", owned_by="calculator", category="math")
def add_numbers(x: int, y: int) -> int:
    return x + y


@agent(
    "calculator",
    provider="handler-only",
    model="offline",
    tools=["add_numbers"],
    auto_discover_tools=False,
    auto_broadcast=False,
)
def calculate(spore):
    results.append(add_numbers(2, 3))


start_agents(calculate, initial_data={"type": "calculate"})
reef = get_reef()
assert reef.wait_for_completion(timeout=5)
assert results == [5]
reef.shutdown()
```

This offline example calls the function directly from the handler. A real model tool loop sends the JSON Schema declaration to a supported provider and executes the handler only after the model returns a complete tool call.

7.2 External `ToolSpec` registration

Use `ToolSpec` when a schema comes from another system. The handler must accept the declared keyword arguments. Async-only handlers require `agenerate()` or `astream()`.

```
from praval import Agent, ToolSpec


spec = ToolSpec(
    name="lookup_record",
    description="Look up one record by identifier.",
    parameters={
        "type": "object",
        "properties": {"record_id": {"type": "string"}},
        "required": ["record_id"],
    },
)

agent = Agent("lookup", provider="openai", model="gpt-5.4-mini")
agent.add_tool_spec(
    spec,
    lambda record_id: {"record_id": record_id, "status": "found"},
)
agent.close()
```

7.3 Durable HITL

HITL applies to model-generated protected tool calls. Praval stores the pending intervention and provider-neutral continuation in SQLite. Approval, argument editing, and rejection are explicit decisions. A resumed process must register compatible tools and provider configuration.

Human review is an interaction design problem as well as a storage mechanism. The reviewer needs enough context to understand the proposed action and its effect (Amershi et al. 2019).

```
from praval import Agent, InterventionRequired, ToolSpec


def critical_write(resource: str) -> str:
    return f"write-complete:{resource}"


spec = ToolSpec(
    name="critical_write",
    description="Write one controlled resource.",
    parameters={
        "type": "object",
        "properties": {"resource": {"type": "string"}},
        "required": ["resource"],
    },
    requires_approval=True,
    risk_level="critical",
    approval_reason="This operation changes an external resource.",
)

agent = Agent(
    "operator",
    provider="openai",
    model="gpt-5.4-mini",
    hitl_enabled=True,
    hitl_db_path="./interventions.sqlite3",
)
agent.add_tool_spec(spec, critical_write)

try:
    agent.generate("Use critical_write for production/orders.")
except InterventionRequired as pending:
    agent.approve_intervention(
        pending.intervention_id,
        reviewer="oncall",
        edited_args={"resource": "staging/orders"},
    )
    print(agent.resume_run(pending.run_id))
finally:
    agent.close()
```

The maintained live HITL certification checks approval, editing, rejection, and cross-process resume. The book does not execute that paid provider path.

7.4 MCP tools

Praval 0.8.1 provides an async tools-only MCP client for stdio and Streamable HTTP. Install `praval[mcp]` on Python 3.10 or newer. Core Praval continues to support Python 3.9.

The client discovers tools, prefixes names, preserves input schemas, registers async handlers, applies approval by default, bounds time and result size, and closes the session or subprocess. The exercised transport revision is the MCP 2025-11-25 specification (contributors 2025).

```
import asyncio

from praval import Agent
from praval.mcp import MCPClient, MCPServerConfig


async def main() -> None:
    agent = Agent(
        "researcher",
        provider="openai",
        model="gpt-5.4-mini",
        hitl_enabled=True,
    )
    config = MCPServerConfig(
        name="notes",
        transport="stdio",
        command="python",
        args=["servers/notes_server.py"],
    )
    try:
        async with MCPClient(config) as client:
            tools = await client.register_tools(agent)
            print([tool.name for tool in tools])
            response = await agent.agenerate("Summarize the notes.")
            print(response.content)
    finally:
        agent.close()


asyncio.run(main())
```

MCP resources, prompts, server hosting, managed OAuth, sampling, elicitation, tasks, legacy SSE, automatic reconnect, and binary result blocks are not supported in 0.8.1. Provider-hosted MCP descriptors are a separate, experimental provider feature.

Chapter 8

Memory, embeddings, storage, and references

Memory and storage solve different problems. Memory selects prior information for an Agent. Storage reads and writes application data through configured providers. `EmbeddingRuntime` creates vectors. `DataReference` lets a Spore point to stored data without carrying the data itself.

These features are optional. Installation, service credentials, schema management, retention, and backup remain deployment responsibilities.

8.1 Memory layers

Praval exposes short-term, episodic, semantic, and long-term memory paths. The names describe retrieval roles in the framework. They are not claims that the implementation reproduces human cognition.

Memory-enabled Agents need a configured backend and compatible embedding settings. A model change or dimension change may require re-indexing. Retrieval quality must be evaluated with application data.

```
from praval.memory import MemoryManager, MemoryQuery, MemoryType


manager = MemoryManager(agent_id="reviewer", backend="memory")
manager.store(
    "The release requires exact-wheel validation.",
    memory_type=MemoryType.SEMANTIC,
    importance=0.9,
)
results = manager.retrieve(
    MemoryQuery(
        query_text="How is the release validated?",
        memory_types=[MemoryType.SEMANTIC],
        limit=3,
    )
)
print(results)
```

The in-memory backend is useful for deterministic tests. Persistent or vector backends have their own dependencies and operating requirements.

8.2 Storage providers

The storage API is asynchronous. Praval includes filesystem, PostgreSQL, Redis, S3-compatible, and Qdrant providers. Service-backed providers need their client libraries and reachable services. The paper-validation suite checks functional round trips against pinned service images. It does not claim durability, failover, or production throughput.

```
import asyncio
import tempfile

from praval.storage import FileSystemProvider


async def roundtrip() -> None:
    with tempfile.TemporaryDirectory() as directory:
        provider = FileSystemProvider(
            "book-files",
            {"base_path": directory},
        )
        try:
            stored = await provider.store(
                "state/value.json",
                {"status": "ready"},
            )
            restored = await provider.retrieve("state/value.json")
            assert stored.success and restored.success
            assert restored.data == {"status": "ready"}
        finally:
            await provider.disconnect()


asyncio.run(roundtrip())
```

There is no hidden cross-provider fallback. Applications that need fallback must define which errors permit it and how consistency is maintained.

8.3 DataReference

A `DataReference` contains a provider name, storage type, resource identifier, and optional metadata. `to_uri()` uses `provider://storage_type/resource_id`. `from_uri()` also accepts the older `praval://storage_type/provider/resour` form.

```
from praval.storage import DataReference, StorageType


reference = DataReference(
    provider="evidence-files",
    storage_type=StorageType.FILE_SYSTEM,
    resource_id="runs/42/report.json",
)
uri = reference.to_uri()
restored = DataReference.from_uri(uri)

assert uri == (
    "evidence-files://file_system/runs/42/report.json"
)
assert restored.provider == "evidence-files"
```

```
assert restored.storage_type is StorageType.FILE_SYSTEM
assert restored.resource_id == "runs/42/report.json"
```

The URI is an application reference. Access control, expiry, integrity, and object retention still belong to the storage system and deployment.

Chapter 9

Observability, failure boundaries, and secure communication

Operations need explicit ownership. Traces help diagnose behavior, but they do not correct an invalid workflow. Encryption protects configured messages, but it does not distribute trust. A transport moves bytes, but it does not define the application's retry policy.

9.1 Observability

Praval provides tracing with console inspection, SQLite storage, and OTLP HTTP export. The implementation finalizes a span before storing it once. Sync and async instrumentation share the same trace model. OTLP integration follows the supported OpenTelemetry path (Authors 2026).

Install the observability extra when the deployment needs its optional dependencies. Configure sampling, storage, and export before creating the workload.

```
from praval.observability import SpanKind, get_tracer


tracer = get_tracer()
with tracer.start_as_current_span(
    "book.review",
    kind=SpanKind.INTERNAL,
) as span:
    span.set_attribute("document.id", "draft-42")
    span.set_attribute("review.status", "accepted")

print(span.trace_id)
```

Trace attributes may contain sensitive data. Record identifiers and bounded metadata rather than prompts, secrets, or full tool results unless the deployment has an explicit data policy.

9.2 Failure boundaries

Praval 0.8.1 does not add a universal circuit breaker, retry every exception, or reconnect every MCP session. Important failure boundaries include:

- Provider capability errors occur before provider execution when the profile declares the requested feature unsupported.
- Provider network and service errors remain provider or transport errors.

- A Reef handler exception completes that scheduled handler with an error. The application still decides whether and how to retry domain work.
- A completion timeout reports that work did not finish within the bound. It does not cancel every underlying external operation.
- A pending HITL run remains dependent on its SQLite database and compatible tool registration.
- Storage providers return `StorageResult` values for documented operations, but backend durability depends on the service.

Use correlation IDs, idempotency keys, bounded timeouts, terminal messages, and explicit cleanup. Test duplicates, late messages, cancellation, service loss, and partial output.

9.3 Secure Spores

The optional secure subsystem uses PyNaCl primitives for authenticated public-key encryption and Ed25519 signatures. Tests cover round trips, tampering, wrong keys, expiry, serialization, and payload-size overhead. These tests are behavioral checks, not cryptographic proofs (Bernstein et al. 2011, 2012; Bernstein 2006).

```
from praval.core.secure_spore import SporeKeyManager


sender = SporeKeyManager("sender")
recipient = SporeKeyManager("recipient")
knowledge = {"type": "controlled_message", "value": 42}

ciphertext, nonce, signature = sender.encrypt_and_sign(
    knowledge,
    bytes(recipient.public_key),
)
restored = recipient.decrypt_and_verify(
    ciphertext,
    nonce,
    signature,
    bytes(sender.public_key),
    bytes(sender.verify_key),
)
assert restored == knowledge
```

The implementation does not provide a production key distribution or rotation protocol. Its long-lived box keys do not establish perfect forward secrecy. Deployments must define identity verification, key storage, revocation, rotation, compromise response, and transport security.

9.4 External transports

AMQP, MQTT, and STOMP adapters are part of the optional secure transport subsystem. They are not interchangeable with the core in-memory Reef. Each transport has broker, authentication, TLS, acknowledgement, ordering, and backpressure settings that require separate tests.

Do not infer horizontal capacity from the presence of a network transport. Measure the actual topology, broker, payload sizes, consumers, and failure policy.

Chapter 10

Test behavior, measure the critical path, and deploy within limits

Framework examples answer whether an API can be used. Experiments answer whether a stated behavior occurs under a defined protocol. Performance claims need repeated measurements, raw samples, environment records, and limits.

10.1 Exact-wheel validation

The research harness verifies the wheel hash, package version, installed import path, source tag, environment, and dirty-tree state. It keeps raw samples separate from summaries and retains failed runs.

Run the offline paper suite with:

```
venv/bin/python -m research.paper_validation run \
  --tier offline \
  --wheel dist/praval-0.8.1-py3-none-any.whl
```

Audit and validate this book with:

```
venv/bin/python -m research.paper_validation book-audit \
  --book docs/archive/praval-book.md

venv/bin/python -m research.paper_validation book-validate \
  --book docs/archive/praval-book.md \
  --wheel dist/praval-0.8.1-py3-none-any.whl
```

Offline evidence uses deterministic providers and local fixtures. Service evidence uses pinned RabbitMQ, PostgreSQL, Redis, MinIO, Qdrant, and OTLP containers. Live evidence is optional and records the provider and model conditions.

10.2 Critical-path performance

Parallel scheduling can shorten a workload only when branches are independent. For independent work with durations (d_1, d_2, …, d_n), the ideal sequential work is their sum, while the ideal parallel critical path is their maximum. Real execution adds scheduling, serialization, transport, and join overhead.

The Praval paper tests sequential, fan-out and fan-in, pipeline, and request or reply shapes at defined branch counts and deterministic work durations. It reports repeated samples and confidence intervals using systems benchmarking guidance (Kalibera and Jones 2013). The result is scoped to those workloads.

Do not claim a universal speedup. Dependent work, short tasks, shared resources, provider rate limits, and model variability can remove the advantage.

10.3 Reef scaling and abstraction costs

Measure throughput and latency separately for in-memory and RabbitMQ paths. Record agent count, payload size, delivery failures, queue depth, CPU, memory, and slow-consumer behavior. Broker results apply to the tested broker image, hardware, and configuration.

Measure framework layers separately:

- Direct adapter call compared with `ModelRuntime`.
- Direct function compared with MCP stdio or Streamable HTTP.
- Observability disabled compared with enabled.
- Plain Spore compared with Secure Spore.

These measurements describe overhead. They do not rank application quality.

10.4 Controlled framework comparison

A controlled comparison pins each framework in a separate environment and uses the same task, context, model work, output constraints, and scoring rule. The Praval harness includes Praval 0.8.1, LangGraph 1.2.9, and CrewAI 1.15.6 for one two-stage task (Sampathkumar 2026c; LangChain 2026; CrewAI 2026).

The deterministic track isolates framework and coordination overhead with a delayed model proxy. A live track is valid only when provider credentials and the requested model are available. Correctness uses expected facts and numbers, not an LLM judge.

Earlier Praval comparison numbers used unequal model-call patterns and are not evidence for 0.8.1. Do not repeat them as current benchmark results.

10.5 Deployment boundaries

Before deployment, record:

- The exact Praval wheel and dependency lock.
- Provider and model profiles, endpoint choices, and capability assumptions.
- Tool schemas, approval policy, credentials, and network access.
- Reef topology, terminal conditions, timeouts, and idempotency rules.
- Memory and embedding model, dimensions, collection identity, and rebuild plan.
- Storage schemas, retention, backup, recovery, and service ownership.
- Trace sampling, redaction, storage, export, and access controls.
- Key distribution, rotation, revocation, and compromise response for secure communication.
- Cleanup ownership for Agents, Reef, MCP clients, storage providers, and observability exporters.

Praval provides components for these concerns. Their presence does not establish production readiness for a particular application.

10.6 Threats to validity

Deterministic providers do not reproduce live provider behavior. Service containers do not reproduce every managed deployment. One hardware environment does not establish universal scaling. A two-stage comparison does not characterize every framework feature. Security behavior tests do not replace protocol review or cryptographic analysis.

Keep negative results. Narrow the claim when evidence covers only one provider, model, topology, or service. Re-run live checks when an external API or model changes.

Chapter 11

Appendices

11.1 Appendix A. Current API map

The following map lists the main 0.8.1 entry points. It is a guide, not a replacement for generated API documentation.

Concern	Main entry points	Result or responsibility
Direct model call	`Agent.chat`, `generate`, `agenerate`	String or `ModelResponse`
Streaming	`Agent.stream`, `astream`	`ModelEvent` sequence
Coordination	`agent`, `broadcast`, `start_agents`	Decorated handler workflow
Delivery	`Reef.send`, `broadcast`, `request`, `reply`	Spore identifier
Completion	`Reef.wait_for_completion`	Boolean completion result
Lifecycle	`Agent.close`, `Reef.shutdown`, `PravalApp`	Explicit cleanup
Embeddings	`EmbeddingRuntime.embed`, `aembed`	`EmbeddingResponse`
Tools	`tool`, `Agent.add_tool_spec`	Registered typed handler
HITL	Agent intervention and resume methods	Durable decision state
MCP	`MCPClient`, `MCPServerConfig`	Async external tool client
Storage	provider classes, `DataManager`	Async `StorageResult`
References	`DataReference`	Portable storage URI
Observability	`get_tracer`, `SQLiteTraceStore`, `OTLPExporter`	Trace records and export

Top-level imports are listed in `src/praval/__init__.py`. Optional imports can be unavailable when their extras are not installed.

11.2 Appendix B. Certified learning path

Use the shipped notebooks and examples in this order:

1. Architecture notebook for Agent, Reef, Spore, and lifecycle.
2. Hello-world notebook for decorated Agents and completion.
3. Research pipeline and parallel Agents for topologies.
4. Tool use, memory, and Qdrant for optional capabilities.
5. Production features for bounded transport, security, and observability examples.
6. ModelRuntime, HITL, MCP, and voice and multimodal for the 0.8 execution plane.

The deterministic certificate is `examples/certification/offline_framework.py`. Service checks are in `examples/certification/services.py`. Live provider, HITL, MCP, voice, and multimodal checks remain separate because their requirements differ.

11.3 Appendix C. Installation and configuration

Install the core wheel:

```
python -m pip install praval==0.8.1
```

Install only the optional capabilities in use:

```
python -m pip install 'praval[memory]'
python -m pip install 'praval[storage]'
python -m pip install 'praval[secure]'
python -m pip install 'praval[pdf]'
python -m pip install 'praval[observability]'
python -m pip install 'praval[mcp]'
```

MCP requires Python 3.10 or newer. The core supports Python 3.9 through 3.13. Provider SDKs and service clients may impose additional constraints.

Common configuration names include:

Setting	Purpose
`PRAVAL_DEFAULT_PROVIDER`	Default provider when Agent does not set one
`PRAVAL_DEFAULT_MODEL`	Default model or compact provider and model value
`OPENAI_API_KEY`	OpenAI authentication
`ANTHROPIC_API_KEY`	Anthropic authentication
`COHERE_API_KEY`	Cohere authentication
`GEMINI_API_KEY`	Gemini authentication
`PRAVAL_HITL_DB_PATH`	Default SQLite HITL path
`PRAVAL_OBSERVABILITY`	`auto`, `on`, or `off`
`PRAVAL_TRACES_PATH`	SQLite trace path
`PRAVAL_OTLP_ENDPOINT`	OTLP HTTP endpoint
`PRAVAL_SAMPLE_RATE`	Trace sample rate

Do not place credentials in `provider_options`, Spore metadata, prompts, traces, or committed configuration.

11.4 Appendix D. Troubleshooting

`No LLM provider credentials found` means the Agent has no explicit provider and no supported provider environment variable. Set an explicit provider and model, then configure its credential or local endpoint.

`HITLConfigurationError` means a protected tool was selected by an Agent that does not have HITL enabled. Enable HITL deliberately, or remove the approval requirement only after a policy review.

An unresolved pending intervention usually means the resuming process is using a different SQLite path, has not registered the Agent and tool, or has not recorded a decision.

A capability error means the active profile does not advertise the requested feature. Confirm the provider, model, endpoint, and profile. Do not bypass the check without testing the endpoint.

`wait_for_completion()` returning `False` means the timeout elapsed while Reef still tracked work. Check for a slow handler, a handler waiting on external I/O, an unbounded broadcast loop, or a missing terminal condition.

An MCP URL error means remote plain HTTP, embedded credentials, or mixed transport fields were rejected. Use HTTPS for remote servers. Plain HTTP is allowed only for loopback development.

An embedding mismatch means the provider, model, or dimensions differ from the indexed collection. Rebuild or select the compatible collection.

11.5 Appendix E. Version history

Book Edition 1.0, October 2025, described Praval 0.7.6. It is superseded by this edition.

Book Edition 1.1, July 2026, describes Praval 0.8.1. It adds the 0.7.22 baseline, the two-plane architecture, current model and embedding contracts, Spore V2, MCP, durable HITL, lifecycle ownership, exact-wheel validation, and bounded claims.

Praval 0.7.22 remains the last stable 0.7 release. Praval 0.8.1 is the first supported 0.8 release. A temporary 0.8.0 upload was withdrawn before a matching tag and release were created, so it is not a supported migration source (Sampathkumar 2026b).

11.6 Appendix F. Build this book

From the Praval repository:

```
venv/bin/python -m research.paper_validation build-book \
  --book docs/archive/praval-book.md \
  --output docs/archive/praval-book.pdf
```

The command audits citations, links, version statements, strong claims, and Python markers before invoking Pandoc and XeLaTeX. Temporary build files stay outside the repository. The final PDF remains beside this Markdown source.

References

Agha, Gul. 1986. *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press. https://mitpress.mit.edu/9780262511414/actors/.

Amershi, Saleema, Dan Weld, Mihaela Vorvoreanu, et al. 2019. "Guidelines for Human-AI Interaction." *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. https://doi.org/10.1145/3290605.3300233.

Authors, OpenTelemetry. 2026. *OpenTelemetry Specification and OpenTelemetry Protocol*. Versions OpenTelemetry 1.59.0; OTLP 1.11.0. https://opentelemetry.io/docs/specs/.

Bernstein, Daniel J. 2006. "Curve25519: New Diffie-Hellman Speed Records." *Public Key Cryptography – PKC 2006*. https://doi.org/10.1007/11745853/_14.

Bernstein, Daniel J., Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. 2012. "High-Speed High-Security Signatures." *Journal of Cryptographic Engineering 2, 77–89*, ahead of print. https://doi.org/10.1007/s13389-012-0027-1.

Bernstein, Daniel J., Tanja Lange, and Peter Schwabe. 2011. *The Security Impact of a New Cryptographic Library*. IACR Cryptology ePrint Archive, Report 2011/646. https://eprint.iacr.org/2011/646.

Broadcom. 2024. *RabbitMQ 3.13 Documentation*. V. 3.13.7. Released. https://www.rabbitmq.com/docs/3.13.

Committee, OASIS AMQP Technical. 2012. *OASIS Advanced Message Queuing Protocol Version 1.0*. Version 1.0. https://docs.oasis-open.org/amqp/core/v1.0/amqp-core-overview-v1.0.html.

contributors, Model Context Protocol. 2025. *Model Context Protocol Specification: Transports*. Versions 2025-11-25. https://modelcontextprotocol.io/specification/2025-11-25/basic/transports.

CrewAI, Inc. 2026. *CrewAI 1.15.6*. V. 1.15.6. Released. https://pypi.org/project/crewai/1.15.6/.

Eugster, Patrick Th., Pascal A. Felber, Rachid Guerraoui, and Anne-Marie Kermarrec. 2003. "The Many Faces of Publish/Subscribe." *ACM Computing Surveys 35(2), 114–131*, ahead of print. https://doi.org/10.1145/857076.857078.

Hewitt, Carl, Peter Bishop, and Richard Steiger. 1973. "A Universal Modular ACTOR Formalism for Artificial Intelligence." *Proceedings of the 3rd International Joint Conference on Artificial Intelligence, 235–245*. https://www.ijcai.org/Proceedings/73/Papers/027B.pdf.

Kalibera, Tomas, and Richard Jones. 2013. "Rigorous Benchmarking in Reasonable Time." *Proceedings of the 2013 International Symposium on Memory Management, 63–74*. https://doi.org/10.1145/2464157.2464160.

LangChain, Inc. 2026. *LangGraph 1.2.9*. V. 1.2.9. Released. https://pypi.org/project/langgraph/1.2.9/.

Sampathkumar, Rajesh. 2026a. *Praval 0.7.22 Release Notes and Source Tag*. V. 0.7.22. Released. https:

//github.com/aiexplorations/praval/releases/tag/v0.7.22.

Sampathkumar, Rajesh. 2026b. *Praval 0.8.1 Release Notes and Source Tag*. V. 0.8.1. Released. https://github.com/aiexplorations/praval/releases/tag/v0.8.1.

Sampathkumar, Rajesh. 2026c. *Praval 0.8.1*. V. 0.8.1. Released. https://pypi.org/project/praval/0.8.1/.

Wright, Austin, Henry Andrews, Ben Hutton, and Greg Dennis. 2022. *JSON Schema Validation: A Vocabulary for Structural Validation of JSON*. Versions Draft 2020-12. https://json-schema.org/draft/2020-12/json-schema-validation.

Yao, Shunyu, Jeffrey Zhao, Dian Yu, et al. 2023. "ReAct: Synergizing Reasoning and Acting in Language Models." *International Conference on Learning Representations*. https://openreview.net/forum?id=WE/_vluYUL-X.